

Structural Maintenance and Repair Verification in Autonomous Orbital Maintenance Platform

Chiara Masaniello

Department of Informatics, Bioengineering, Robotics and Systems Engineering

University of Genoa

Genova, Italy

s8434670@studenti.unige.it

Abstract—This report presents a planning model for an Autonomous Orbital Maintenance Platform using PDDL and its continuous-time extension, PDDL+. The study models a free-climbing robot executing a causal workflow of inspection, diagnosis, repair, and verification on aerospace components. While the standard PDDL formulation establishes static logical dependencies, the PDDL+ model introduces temporal dynamics, continuous physical degradation, and cascading failures. The research evaluates the impact of deferred interventions on mission feasibility, integrates continuous repair mechanisms.

Index Terms—Autonomous Maintenance, PDDL, PDDL+, Temporal Planning, Continuous Processes, Causal Dependencies, Combinatorial Explosion, Aerospace Robotics.

I. PROJECT OVERVIEW

The project aims to develop a PDDL/PDDL+ model for the maintenance and structural repair of an autonomous orbital maintenance platform. The structure of the model focuses on defining a domain for a free-climbing robot that follows a precise causal workflow: inspection, diagnosis, repair, and verification of the component.

The assignment considers two components: the antenna-bracket and the radiator. For the initial PDDL modeling (Q1 model), two types of damage were implemented for each component: loose and cracked for the antenna, and coolant leak and structural deformation for the radiator. These concepts were subsequently revisited and expanded in PDDL+ (Q2 model). Specifically, for the antenna, the loose and cracked states are connected through causality, culminating in a total failure state if left unrepaired. For the radiator, the same causal logic is applied to the structural deformation, which is preceded by an intermediate thermal bowing condition and can likewise escalate into failure if the degradation is not mitigated in time.

The remainder of this report is organized as follows. Section III presents the Q1 domain and the two problem instances developed for it. Section IV extends the model into PDDL+, introducing continuous degradation processes and discrete events. Section V discusses the modeling choices, the differences between the two formulations, and the limitations of the approach.

II. DESIGN LOGIC

The model follows a logical workflow in PDDL where the robot must first use a sensing tool (e.g., camera) to reveal the symbolic diagnosis, then apply the correct tool (e.g., torque wrench or thermal tape), and finally perform a mandatory verification action before achieving the goal. In fact, the goal is not to model detailed rigid body equations, but to find symbolic or hybrid abstractions. In other words, the goal is to use Hybrid Abstraction, specifically the use of piecewise linearization (e.g., degradation rates that accelerate in steps via :event) to prevent the planning engine from crashing, while maintaining physical fidelity.

The use of the staged workflow **uninspected - inspected - nominal/degraded - repaired - verified** also allows focusing on the repair process specific to each component, enabling an association between symbolic modeling and real modeling.

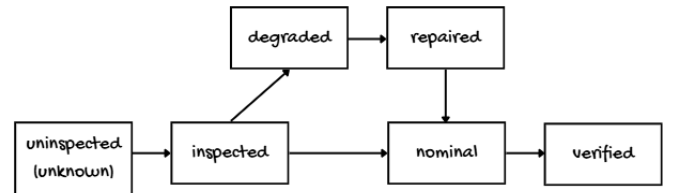


Fig. 1: Architecture design

A. Bracket Antenna

The *bracket antenna* is the structural support that fixes the antenna to the body of the orbital platform. It must ensure the perfect alignment of the antenna for communications. Thermal variations and mechanical stress can lead to loosening of the fastening screws or, in worse cases, deform the bracket or create micro-cracks due to material fatigue [1]. The diagnosis of the antenna is carried out by a visual camera capable of detecting misalignments, structural breaks, or bolt displacements [2].

In PDDL, individual damages are addressed:

- *is-loose* (loosening) is repaired with a simple tightening, so the robot will need to use a single tool, the *torque wrench*.

- *cracked – bracket* (misalignments/breaks) requires a full replacement, so the robot will need to use a *grasping – tool* and carry a *spare – bracket* with it.

For the assignment, it was decided to consider both tools and sensors as well as spare parts that can be carried in the robot’s slots. Consequently, the highly complex operations required for the actual replacement of large Orbital Replaceable Units (ORUs), typically validated through extensive ground mock-ups, are considered beyond the current scope [3].

Instead, in PDDL+, these two actions are correlated; the phase error increases proportionally with time, amplified by mechanical stresses [4]. Rather than associating this with the vacuum of space, literature confirms that space antennas are highly flexible and lightweight structures, making them extremely vulnerable to transient disturbances, such as vibration levels induced by the robot’s locomotion [1] [5]. Exceeding a threshold, the antenna becomes loose (*is – loose*); if not repaired, it worsens until it breaks (*cracked – bracket*), and in the worst-case scenario, it can lead to the total failure of the mission [6].

B. Radiator

Used to maintain the station at operational temperature. Through the circulation of a refrigerant, generally anhydrous ammonia, it absorbs heat from the electronic components and radiates it into space, thereby controlling the temperature of the operating environment [7]. The diagnosis of the component is carried out using a 3D profilometer scanner capable of analyzing the surface to detect craters, scratches, or impact deformations.

In PDDL, the following damages are addressed:

- *coolant – leak*, that is, a leak in the coolant circuit, generally due to the presence of hypervelocity micrometeoroids and orbital debris (MMOD) [8]–[10]. To repair it, it is therefore necessary to isolate and repair the damaged panel using the *coolant – bypass – tool* and add coolant (considered by the robot as a spare part).
- *structural – deformation* arises from the difference in materials present in the component; the different thermal coefficients react differently to thermal shocks [11]. Thermal stress, aided by the degradation of the emissivity paint used on the component, can lead to ratcheting and bowing of the material [12]. To repair these deformations, a 3D printing process is used, meaning the robot deposits polymeric or metallic material layer by layer to fill the crater and restore the surface continuity. The assignment therefore considers the use of the *additive – extruder* tool and the ‘printed’ material (which the parser always considers as a spare part).

In PDDL+, we model the continuous environmental degradation. The radiator paint progressively deteriorates over

time due to solar exposure, increasing the component’s solar absorptivity [12]. When this degradation exceeds a critical threshold, it triggers an event that significantly increases the speed of the structural thermal stress (*strain_rate*). If the process is not mitigated using *thermal – tape* to restore thermal properties and stop the overheating [8], the accumulated stress will eventually push the component into a *component – bowed* state, causing an irreversible structural deformation that can lead to mission failure.

III. Q1 – BASIC PDDL MODEL

The implementation of the initial logical model in standard PDDL aims to validate the causal dependencies of the domain. At this stage, time is not considered, and actions produce instantaneous effects. The objective is to demonstrate the ability of the planning engine to manage the strict sequence of operations required for maintenance and to correctly manage the inventory of the robot through slots, respecting the physical limits of the platform.

Note on Domain Coverage

Before addressing the issues related to PDDL, let’s take a closer look at the domain used to solve them.

TABLE I: Q1 - Domain Components and Logical Mapping

Parameter	antenna-bracket	radiator
Damages	is-loose cracked-bracket	coolant-leak structural-deformation
Sensors	visual-camera	profilometer
Tools & Parts	torque-wrench grasping-tool, spare-bracket	bypass, creserver additive-extruder
Resolution	Tighten (loose) Replace (cracked)	Bypass and Refill (leak) Print material (deformation)

Although the domain defines a dedicated repair action for the structural deformation of the radiator (*structural-deformation-reparation*), this specific damage type is not exercised in the Q1 problem instances. To avoid redundancy and comprehensively showcase the solver’s capabilities across different scenarios, the Q1 instances focus exclusively on the remaining three repair paths (antenna tightening, antenna replacement, and coolant leak bypass). The structural deformation scenario is intentionally reserved for the Q2 continuous model, where its progressive nature and the prolonged printing repair process can be more accurately simulated.

A. Instance 1: Simple Degradation

The first instance, described in the file *Q1_problem1.pddl*, defines a basic scenario to test the linear logical flow. The environment consists of a tool storage (*esp-storage*), a docking port from where the robot departs (*docking-port*), and the intervention site (*antenna-site*).

The damaged component is a single antenna (*antennal*) that is in the *is-loose* state. The robot has three free

slots, a quantity more than sufficient to carry the necessary equipment: the camera (`cam1`) for diagnosis and the torque wrench (`twrench1`) for repair.

The generated plan, shows an interesting behavior of the search engine. Instead of pre-filling the slots, the robot performs the operations by shuttling between the warehouse and the component: it first picks up the camera, goes to the site, inspects (`inspect-component`), and diagnoses the damage (`degraded-diagnosis`). Subsequently, it returns to the warehouse, deposits the sensor (`store-equipment`), and retrieves the torque wrench to go back to the antenna and perform the repair (`loose-antenna-reparation`). Finally, it places the tool in the slot, makes one last trip to retrieve the camera again, and performs the mandatory verification action (`repaired-verify`), thus achieving the goal.

```
;;!domain: Q1
;;!problem: Q1-problem1

0.00000: (MOVE R DOCKING-PORT ESP-STORAGE)
0.00100: (UNSTORE-EQUIPMENT R ESP-STORAGE CAM1)
0.00200: (MOVE R ESP-STORAGE ANTENNA-SITE)
0.00300: (INSPECT-COMPONENT R ANTENNA1 ANTENNA-SITE CAM1)
0.00400: (DEGRADED-DIAGNOSIS ANTENNA1 IS-LOOSE CAM1)
0.00500: (MOVE R ANTENNA-SITE ESP-STORAGE)
0.00600: (STORE-EQUIPMENT R ESP-STORAGE CAM1)
0.00700: (UNSTORE-EQUIPMENT R ESP-STORAGE TWRENCH1)
0.00800: (MOVE R ESP-STORAGE ANTENNA-SITE)
0.00900: (LOOSE-ANTENNA-REPARATION R ANTENNA-SITE ANTENNA1 TWRENCH1)
0.01000: (PUT-IN-SLOT R TWRENCH1 SLOT3)
0.01100: (MOVE R ANTENNA-SITE ESP-STORAGE)
0.01200: (UNSTORE-EQUIPMENT R ESP-STORAGE CAM1)
0.01300: (MOVE R ESP-STORAGE ANTENNA-SITE)
0.01400: (REPAIRED-VERIFY R ANTENNA1 ANTENNA-SITE CAM1)

; Makespan: 0.014000000000000005
; Metric: 0.014000000000000005
```

B. Instance 2: Complex Scenario with Limited Resources

The second instance (`Q1_problem2.pddl`) introduces a significantly higher level of complexity, testing the system's ability to plan in the face of limited resources and multiple simultaneous damages.

In this scenario, the robot must deal with two compromised components located in different positions: the antenna that has the cracked-bracket damage (requiring a physical replacement) and the radiator (`radiator1`) affected by a coolant-leak (requiring a bypass and a fluid refill). The robot has only three physical slots, but the resolution requires the use of six distinct elements (in addition to managing the antenna waste). Since it is impossible to transport everything in a single trip, the planner develops a logistical strategy with interleaved operations, as shown in the following plan. The system does not solve one component at a time, but optimizes movements and inventory:

- 1) It performs the antenna diagnosis, then stows the camera in its inventory (`put-in-slot R cam1 slot1`) to avoid having to return it to the warehouse.
- 2) It retrieves the profilometer to diagnose the damage to the radiator.

- 3) It loads the bypass and the fluid reserve (`creserver1` in `slot3`), performs the radiator repair, and verifies it with the profilometer.
- 4) It dedicates itself to replacing the antenna by taking the gripping tool and the spare bracket (`spare-bracket1` in `slot2`), repairs the component, and extracts the camera from slot 1, which has been kept since the beginning, for the final validation.

The entire process demonstrates how the sequence of actions strictly follows the cycle *Move* → *Inspect* → *Diagnose* → *Repair* → *Verify*, dynamically adapting to the robot's spatial constraints.

```
;;!domain: Q1
;;!problem: Q1-Testingproblem

0.00000: (MOVE R DOCKING-PORT ESP-STORAGE)
0.00100: (UNSTORE-EQUIPMENT R ESP-STORAGE CAM1)
0.00200: (MOVE R ESP-STORAGE ANTENNA-SITE)
0.00300: (INSPECT-COMPONENT R ANTENNA1 ANTENNA-SITE CAM1)
0.00400: (DEGRADED-DIAGNOSIS ANTENNA1 CRACKED-BRACKET CAM1)
0.00500: (MOVE R ANTENNA-SITE ESP-STORAGE)
0.00600: (PUT-IN-SLOT R CAM1 SLOT1)
0.00700: (UNSTORE-EQUIPMENT R ESP-STORAGE PROFILOMETER1)
0.00800: (MOVE R ESP-STORAGE RADIATOR-SITE)
0.00900: (INSPECT-COMPONENT R RADIATOR1 RADIATOR-SITE PROFILOMETER1)
0.01000: (DEGRADED-DIAGNOSIS RADIATOR1 COOLANT-LEAK PROFILOMETER1)
0.01100: (MOVE R RADIATOR-SITE ESP-STORAGE)
0.01200: (STORE-EQUIPMENT R ESP-STORAGE PROFILOMETER1)
0.01300: (UNSTORE-EQUIPMENT R ESP-STORAGE CRESERVER1)
0.01400: (PUT-IN-SLOT R CRESERVER1 SLOT3)
0.01500: (UNSTORE-EQUIPMENT R ESP-STORAGE CBYPASS1)
0.01600: (MOVE R ESP-STORAGE RADIATOR-SITE)
0.01700: (COOLANT-RADIATOR-REPARATION R RADIATOR-SITE RADIATOR1 CBYPASS1 CRESERVER1 SLOT3)
0.01800: (MOVE R RADIATOR-SITE ESP-STORAGE)
0.01900: (STORE-EQUIPMENT R ESP-STORAGE CBYPASS1)
0.02000: (UNSTORE-EQUIPMENT R ESP-STORAGE PROFILOMETER1)
0.02100: (MOVE R ESP-STORAGE RADIATOR-SITE)
0.02200: (REPAIRED-VERIFY R RADIATOR1 RADIATOR-SITE PROFILOMETER1)
0.02300: (MOVE R RADIATOR-SITE ESP-STORAGE)
0.02400: (STORE-EQUIPMENT R ESP-STORAGE PROFILOMETER1)
0.02500: (UNSTORE-EQUIPMENT R ESP-STORAGE SPARE-BRACKET1)
0.02600: (PUT-IN-SLOT R SPARE-BRACKET1 SLOT2)
0.02700: (UNSTORE-EQUIPMENT R ESP-STORAGE GRASPER1)
0.02800: (MOVE R ESP-STORAGE ANTENNA-SITE)
0.02900: (CRACKED-BRACKET-REPARATION R ANTENNA-SITE ANTENNA1 GRASPER1 SPARE-BRACKET1 SLOT2 ANTENNA1-DEBRIS)
0.03000: (MOVE R ANTENNA-SITE ESP-STORAGE)
0.03100: (STORE-EQUIPMENT R ESP-STORAGE GRASPER1)
0.03200: (MOVE R ESP-STORAGE ANTENNA-SITE)
0.03300: (TAKE-FROM-SLOT R SLOT1 CAM1)
0.03400: (REPAIRED-VERIFY R ANTENNA1 ANTENNA-SITE CAM1)

; Makespan: 0.034000000000000002
; Metric: 0.034000000000000002
```

IV. Q2 – PDDL+ MODEL

The extension of the domain to PDDL+ marks the transition from instantaneous logical modeling to a dynamic physical-temporal environment. At this stage, time becomes a critical factor, and numerical variables (`:numeric-fluents`) allow for continuous tracking of the health status of the components.

A. Continuous Degradation Processes

Environmental and mechanical degradation is no longer considered solely as an initial discrete state, but is actively cal-

culated by the planning engine thru the `:process` constructs. These define constant degradation rates over time, modeling the physics of the domain:

- **degrade_antenna_phase**: a continuous process that increases the phase error (`phase_error`) of the antenna based on the vibration level (`vibration_level`) it is subjected to during its stay in space and the movements of the platform.
- **increased_thermal_strain**: a process that constantly increases the thermal stress of the radiator (`thermal_strain`), based on a predetermined rate of speed (`strain_rate`).

B. Threshold Events and Cascading Failures

The accumulation of continuous damage does not generate discrete states arbitrarily, but triggers a chain reaction based on the surpassing of precise numerical thresholds. To manage this evolution without the active intervention of the robot, the domain uses the `:event` constructs.

TABLE II: Q2 - Continuous Dynamics and Threshold Events

Parameter	antenna-bracket	radiator
Fluents	<code>phase_error</code>	<code>thermal_strain</code> <code>strain_rate</code>
Degradation	<code>degrade_antenna_phase</code>	<code>increased_thermal_strain</code>
Events	≥ 0.1 : <code>is-loose</code> ≥ 0.2 : <code>cracked-bracket</code> ≥ 0.3 : <code>failed</code>	≥ 10.0 : <code>thermal-bowing</code> ≥ 20.0 : <code>structural-def.</code> ≥ 30.0 : <code>failed</code>
Sensors	<code>visual-camera</code>	<code>profilometer</code>
Tools & Parts	<code>torque-wrench</code> <code>grasping-tool</code> , <code>spare-bracket</code>	<code>thermal-tape</code> <code>additive-extruder</code> , <code>print-mat.</code>
Resolution	<code>Tighten (loose)</code> <code>Replace (cracked)</code>	<code>Apply tape (bowing)</code> <code>Continuous print (deform.)</code>

This logical structure defines the progressive worsening of the damage until the irreversible failure (`failed`) of the component:

- **Antenna Bracket**: exceeding the `phase_error` thresholds triggers the event `antenna_becomes_loose` (≥ 0.1). If ignored, the process continues until it triggers `antenna_becomes_cracked` (≥ 0.2) and, in the worst-case scenario, concludes with `antenna_failed` (≥ 0.3).
- **Radiator**: the degradation is accelerated by the environmental event `paint_causes_overheating`, which increases the `strain_rate`. The accumulated thermal stress sequentially activates the events `radiator_starts_bowing` (≥ 10.0), followed by `radiator_structural_failure` (≥ 20.0), culminating in the disastrous `radiator_failed` (≥ 30.0).

```
;;!domain: Q2
;;!problem: Q2-Problem

0: (move R docking-port esp-storage)
0: (move R esp-storage antenna-site)
0: (take-from-slot R slot1 cam1)
0: (inspect-component R antennal antenna-site cam1)
0: (put-in-slot R cam1 slot1)
0: (degraded-diagnosis antennal is-loose cam1)
```

```
0: (take-from-slot R slot2 twrench1)
0: (loose-antenna-reparation R antenna-site antennal
  twrench1)
0: (put-in-slot R twrench1 slot2)
0: -----waiting----- [1.0]
1.0: (take-from-slot R slot1 cam1)
1.0: (repaired-verify R antennal antenna-site cam1)
1.0: (put-in-slot R cam1 slot1)
1.0: (move R antenna-site radiator-site)
1.0: (take-from-slot R slot3 profilometer1)
1.0: (inspect-component R radiator1 radiator-site
  profilometer1)
1.0: (degraded-diagnosis radiator1 structural-deformation
  profilometer1)
1.0: (put-in-slot R profilometer1 slot3)
1.0: (take-from-slot R slot5 aextruder1)
1.0: (start-structural-deformation-reparation R radiator-
  site radiator1 aextruder1 printmat1 slot4)
1.0: -----waiting----- [6.0]
6.0: (put-in-slot R aextruder1 slot5)
6.0: (take-from-slot R slot3 profilometer1)
6.0: (repaired-verify R radiator1 radiator-site
  profilometer1)

; Makespan: 1
; Metric: 1
```

C. Continuous Repair Process

PDDL+ allows overcoming the concept of instantaneous repair by modeling interventions that require prolonged execution over time. In the case of the structural deformation of the radiator, the repair does not occur thru an isolated `:action`, but is managed by the `apply_printed_layers` process.

Once the extruder is started, the robot deposits material over time (`layers_printed`). The intervention is validated and considered complete only when the deposited material equals or exceeds (cautiously) the required amount (`layers_to_print`), a condition that triggers the conclusive event `structural_repair_complete`, restoring the thermal variables to nominal values.

D. Time-Criticality and Infeasible Scenarios

The introduction of continuous processes imposes the concept of a physical *deadline*, forcing the planner to optimize the paths to prevent failure events from triggering before the robot can complete the repairs.

The instance defined in the file `Q2_problemInfeasible.pddl` mathematically demonstrates the impact of the delay on the mission. In the initial state, the radiator is already at a critical level of `thermal_strain` equal to 29.0, accompanied by an accelerated degradation rate (`strain_rate` equal to 2.0). Since the threshold for the `radiator_failed` event is rigidly set at 30.0, the time required for the robot to navigate from the `docking-port`, extract the appropriate tools, and perform the logical sequence of inspection and diagnosis far exceeds the remaining physical time before structural failure. This is further aggravated by the accelerated degradation rate configured in this instance, which is significantly higher than the rate used in the solvable instance (0.25), leaving virtually no operational margin for the robot to intervene.

To explicitly quantify this temporal constraint, Table III compares the mathematical margins of the two instances.

TABLE III: Mathematical Analysis of the Temporal Deadline

Parameter	Q2_Problem	Q2_Infeasible
Initial Strain	25.0	29.0
Strain Rate	0.25	2.0
Fail Threshold	30.0	30.0
Available Margin*	20.0	0.5

* $\text{Margin} [\text{time units}] = (\text{Fail Threshold} - \text{Initial Strain}) / \text{Strain Rate}$

Consequently, the search engine is unable to formulate any valid sequence of actions to prevent failure, explicitly returning the impossibility of resolution.

This scenario proves that, in the presence of temporal dynamics, **delaying inspection interventions renders the mission infeasible**, validating the robustness and physical realism of the implemented model.

```
;;!domain: Q2
;;!problem: Q2-Infeasible

; Makespan: 0
; Metric: 0
; States evaluated: 59
```

E. Degradation Rate and Operational Ordering

In the Q1 model, the sequence in which the robot addresses different components is logically arbitrary: since no notion of time is present, any valid ordering of inspection, diagnosis, and repair actions leads to the goal state. The introduction of continuous degradation in Q2 removes this arbitrariness, since each component accumulates damage independently and at its own rate.

A component with a low degradation rate, such as the radiator in `Q2_problem.pddl` (`strain_rate` equal to 0.25), can be safely addressed after other tasks without risking failure. Conversely, when the same physical quantity is configured with a substantially higher rate, as in `Q2_problemInfeasible.pddl` (`strain_rate` equal to 2.0), the same delay that was previously acceptable becomes the direct cause of mission failure. The comparison between these two instances therefore shows that, under continuous degradation, the ordering of inspection and repair actions is no longer a matter of logical convenience, but a physical constraint imposed by the relative degradation rates of the components involved.

It is worth noting that the domain does not encode any explicit prioritization mechanism: the planner is not guided by a cost function or heuristic that accounts for the degradation rate of each component. The correct ordering, when it exists, emerges implicitly from the search process, constrained by the threshold events defined in the domain. This is an honest limitation of the current model, and it is further discussed in Section VI as a possible direction for future extension.

F. Computational Optimization and Initial State Constraints

Compared to the symbolic model of Q1, the introduction of temporal dynamics and numerical variables in PDDL+

has resulted in a drastic increase in computational load. The increase in the search space has pushed the planning engine to its limits, making some heuristic assumptions necessary to prevent the parser from crashing and to circumvent the problem of combinatorial explosion.

To ensure the convergence of the plan within acceptable times, the initial state (`:init`) of the solvable instance (`Q2_Problem.pddl`) has been optimized under two fundamental aspects:

- **Initial state of the components:** The components do not start from a nominal state and then wait for the natural course of degradation. On the contrary, the instance is initialized with the damage already partially underway. The antenna starts with a phase error already accumulated (`phase_error = 0.15`) which places it in the `is-loose` state, while the radiator is already in the critical phase of structural-deformation (`thermal_strain = 25.0`).
- **Logistics and slot management:** Unlike Q1, where the robot had to autonomously calculate the optimal travel sequences to pick up and exchange tools from the warehouse, in the Q2 model the robot starts the mission with the 5 available slots already equipped with the exact set of tools required for the mission (`cam1`, `twrench1`, `profilometer1`, `printmat1` and `aextruder1`).

These design restrictions do not invalidate the logic of the continuous domain, but act as guiding constraints that reduce the branches of the search tree, allowing the planner to focus computational resources exclusively on the correct scheduling of temporal processes and the validation of failure thresholds.

V. DISCUSSION

The transition from a purely symbolic model (Q1) to a continuous time-driven framework (Q2) highlights several fundamental challenges in modeling autonomous robotic maintenance. The development and comparison of these paradigms offer critical insights into system architecture, diagnostic logic, and the current limitations of automated planning engines.

A. How do causal dependencies structure maintenance planning?

In the symbolic domain, maintenance planning is strictly governed by logical causal dependencies. The operational pipeline is rigidly structured: inspection acts as the logical trigger for diagnosis, which in turn satisfies the preconditions for repair and subsequent verification. The state machine progresses sequentially (`unknown/uninspected` → `inspected` → `degraded` → `repaired` → `verified`), with the initial label differing slightly between Q1 and Q2 for parser compatibility reasons. In the PDDL+ extension, this causality evolves from purely logical to spatio-temporal and physical. Degradation itself becomes an active causal agent: the physical movement of the robot accumulates vibrations that induce the loosening of the antenna, while unmitigated thermal stress causes paint degradation, which recursively accelerates structural deformation.

B. How does symbolic diagnosis differ from physical diagnosis?

A structural limitation of the standard PDDL formulation is the “omniscience” of the planner. By supplying the predicate (`component-damaged antennal is-loose`) in the initial state, the solver inherently knows the absolute truth at $T = 0$. Precondition constraints force the robot to execute the inspection action merely to satisfy the causal chain, even though the hidden information is already algorithmically exposed. The PDDL+ model mitigates this epistemic flaw at the architectural level: the domain does not require the damaged state to be symbolically pre-assigned, since it can instead emerge dynamically as a direct consequence of continuously evolving numerical variables (e.g., `phase_error`). In principle, the planner could compute the actual physical development of the damage over time and diagnose it without prior knowledge.

It should be noted, however, that the problem instances presented in this work still partially pre-assign the damage state at $T = 0$, as discussed in Section IV-F, due to the computational cost of letting the degradation process evolve from a fully nominal state to failure. The architectural capability to derive damage purely from continuous evolution is therefore demonstrated in principle by the domain, rather than fully exercised by the specific instances solved. This physical diagnosis, when actually exercised, is closer to real-world dynamics than the static symbolic assumption of Q1; nonetheless, the domain simplifications required to ensure solvability mean that, in practice, it still retains a partially symbolic nature.

C. How does PDDL+ change the interpretation of deferred repair?

The adoption of PDDL+ fundamentally alters the interpretation of deferred repair. In standard PDDL, the absence of a temporal dimension renders delays inconsequential: a damaged component will indefinitely remain in its static degraded state until the robot intervenes. Conversely, the introduction of `:process` and `:event` constructs gives deferred actions critical physical consequences. As demonstrated in the infeasibility tests, delaying the repair of a highly stressed component (like the radiator) to prioritize another task allows the continuous degradation process to surpass irreversible thresholds (e.g., `thermal_strain` ≥ 30.0). This triggers cascading failure events, inevitably leading to the definitive failure of the mission.

D. How could this model support future autonomous maintenance benchmarks?

The implemented models highlight the current gap between offline planning and online control, providing a baseline for future autonomous maintenance benchmarks. For instance, in Q2, the planner deterministically calculates the exact time required to deposit material (`layers_to_print`). In a real-world robotic application, this process would require a closed-loop feedback system: the action would not last for a pre-

calculated duration but would be dynamically interrupted by onboard sensors once the structural correction is physically verified.

Furthermore, the model exposes the current computational limits of continuous planning engines like ENHSP. A realistic simulation would require the robot to navigate back and forth to the storage module, continuously managing a limited number of slots. However, in a continuous numeric domain, the introduction of multiple objects and the freedom to exchange them arbitrarily at any time triggers a massive combinatorial explosion of the search space, often causing the algorithm to run out of memory. This forced the simplification of the benchmark by pre-equipping the robot, underscoring the pressing need for more efficient search heuristics to handle inventory and logistics in PDDL+ environments.

VI. LIMITATIONS AND FUTURE PERSPECTIVES

While the developed PDDL and PDDL+ models successfully encapsulate the logical and temporal dynamics of autonomous robotic maintenance, they inherently rely on several abstractions. Acknowledging these structural limitations is crucial for contextualizing the validity of the benchmark and charting the course for future physical implementations.

A. Closed-World Assumption and Determinism

The planning domain operates strictly under the Closed-World Assumption: any state or event not explicitly modeled is assumed to be false or non-existent. The environment is completely static outside of the robot’s actions and the predefined degradation processes. Furthermore, the model is entirely deterministic. In real-world space operations, stochastic variables play a major role: sensors might return noisy data, a tool might slip during a repair, or unforeseen external anomalies might occur. The current PDDL framework assumes a 100% success rate for every executed action, lacking mechanisms for probabilistic outcomes or dynamic replanning upon action failure.

B. Kinematic and Spatial Abstraction

The spatial representation of the domain is highly abstracted. Locations (`docking-port`, `antenna-site`) are modeled as discrete nodes in a topological graph, and movement itself remains an instantaneous transition between them in both Q1 and Q2. Time-dependent behavior in the PDDL+ model is instead confined to the continuous processes and events governing degradation and repair, rather than to locomotion itself. The model completely ignores continuous geometry, 3D spatial constraints, and collision avoidance. Moreover, the robotic manipulation is reduced to symbolic slot assignments (`put-in-slot`, `take-from-slot`). In reality, grasping a tool or applying a printed layer requires complex inverse kinematics, precise trajectory planning, and force-torque feedback, none of which can be efficiently represented in pure PDDL.

C. Rigidity of Goal States

Standard PDDL goals are strictly boolean: they are either entirely satisfied or not. There is no concept of "partial success" or reward optimization. If the temporal deadline (e.g., in `Q2_problemInfeasible.pddl`) is too tight to repair both components, the planner simply returns a failure. It cannot autonomously decide to prioritize the most critical component (e.g., the radiator) and sacrifice the antenna to salvage the mission partially. Implementing soft goals or cost-based optimization metrics would be necessary to allow the robot to perform autonomous triage in unresolvable scenarios.

D. Future Perspectives: Beyond Computational Limits

As highlighted by the feasibility tests, pushing the PDDL+ model to encompass more realistic continuous dynamics inevitably leads to a combinatorial explosion of the state space, hitting the hard computational limits of current numeric planners like ENHSP.

Consequently, future developments of this model should not focus on adding more continuous variables or intricate geometric rules within the PDDL syntax itself. Instead, the most promising evolution involves integrating this symbolic model into a hierarchical execution framework (such as ROSPlan or PlanSys2). In such an architecture, the PDDL+ solver would act exclusively as the high-level decision-maker (managing inventory, logical dependencies, and temporal scheduling), while delegating the continuous physical execution, geometric pathfinding, and stochastic anomaly handling to specialized sub-symbolic controllers.

REFERENCES

- [1] H. Djodjodhardjo, "Structural integrity of spacecraft structures subject to motion, thermo-structural dynamics and environmental effects - an overview," *Acta Astronautica*, vol. 222, pp. 219–243, 2024.
- [2] F. Angeletti, P. Iannelli, P. Gasbarri, M. Panella, and A. Rosato, "A study on structural health monitoring of a large space antenna via distributed sensors and deep learning," *Sensors*, vol. 23, p. 368, 2023.
- [3] G. A. Johnson, "The role of mock-ups in the development of orbital replaceable units (oru)," in *Proceedings of the Seminar on Space Station Human Productivity*. NASA Ames Research Center, 1985.
- [4] H. S. C. Wang, "Performance of phased-array antennas with mechanical errors," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 28, no. 2, pp. 535–545, 1992.
- [5] C. Wang, Y. Wang, P. Lian, S. Xue, Q. Xu, Y. Shi, Y. Jia, B. Du, J. Liu, and B. Tang, "Space phased array antenna developments: A perspective on structural design," *IEEE Aerospace and Electronic Systems Magazine*, 2020.
- [6] K. Ueno, "Pattern degradation due to satellite antenna deformation in space orbit," in *Proceedings of ISAP '85*, 1985, pp. 013–3.
- [7] A. J. Juhasz and G. P. Peterson, "Review of advanced radiator technologies for spacecraft power systems and space thermal control," NASA, Technical Memorandum 4555, June 1994.
- [8] E. Christiansen, "Mmod protection and degradation effects for thermal control systems," NASA Johnson Space Center, Tech. Rep., August 2014.
- [9] D. Lear, E. Christiansen, J. Hyde, J. Herrin, F. Lyons, J. Kerr, and S. Ryan, "Investigation of shuttle radiator micro-meteoroid & orbital debris damage," in *50th AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics, and Materials Conference*, Palm Springs, California, 2009.
- [10] J. L. Rhatigan, E. L. Christiansen, and M. L. Fleming, "On protection of freedom solar dynamic radiator from the orbital debris environment: Part i - preliminary analysis and testing," NASA, Technical Memorandum 102458, April 1990.
- [11] S. L. Rickman, "Introduction to on-orbit thermal environments," NASA, Tech. Rep. NF1676L-19094, 2014.
- [12] W. H. Dauterman and L. D. Montgomery, "Design and analysis of the radiator structure for space power systems," Atomics International Division / Rockwell International, Tech. Rep. AI-AEC-13093 / NASA-CR-121222, 1973.